```
Output: (PHP 7.4)
bool(true)
bool(true)
bool(true)

Output: (PHP 8.4)
bool(true)
bool(false)
bool(true)
```

It is best practice to use strongly typed comparison, especially in matters of security, and explicit type casting. Be cautious with type juggling! For more detail on comparison rules, see Chapter 4, "Logic and Control."

Summary

Hopefully you now have a strong grasp of how to make numbers work for you in PHP. We've covered the main pitfalls of numeric computation on silicon. The problems highlighted in this chapter are not PHP's "fault" as such; the limits are generally imposed lower down in the system.

We've seen what happens out there at the edges of 64-bit floating-point arithmetic, or when rational decimals don't have rational binary counterparts, and some strategies to mitigate those problems. The BCMath and GMP extensions can help out greatly when large numbers need precise calculations, and I've signposted other extensions to help any of you working with stats or trading data.

Finally, we looked at PHP's type coercion system in the context of numbers and the potential issues therein. I hope I've made a case for type safety in your future programming adventures.

The open source community is a great place to start if your use case isn't covered here. Linear algebra, complex numbers, matrices, and cryptography (especially) are all areas of interest in some sections of the PHP community. Go forth and enjoy yourselves! The sky's the limit (the sky is not represented by a 64-bit integer).